Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Semester 2 Tahun 2025/2026

Dipersiapkan oleh:

Muhammad Jordan Ferimeison (13524047)

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

1. Desain dan Implementasi

a. Desain

Program dibuat menggunakan algoritma *divide and conquer*. Program akan menerima masukkan berupa path file .obj yang ingin di-*voxelization*, kedalaman (opsional), dan path file .obj untuk output. Program kemudian mengeluarkan hasil pada terminal berupa durasi komputasi, banyaknya voxel yang terbentuk, banyaknya vertices yang terbentuk, banyaknya faces yang terbentuk, banyaknya node yang dijelajahi pada setiap kedalaman, dan banyaknya node yang tidak dijelajahi pada setiap kedalaman. Program dibuat dalam bahasa C#. Bahasa ini menggunakan pendekatan berbasis objek. Desain dimulai dengan mengidentifikasi objek yang berinteraksi dalam proses *voxelization*. Pada program ini, objek yang teridentifikasi adalah Mesh (file .obj yang asli), Octree, Vector, Voxelizer, AABB dan objek bantuan seperti ObjectIO dan Program.

Pada program ini, objek akan membagi dirinya menjadi "octree" yang lebih kecil. Hal ini sejalan dengan konsep "divide" dalam divide and conquer, dimana penyelesaian masalah dilakukan setelah melakukan dekomposisi masalah menjadi masalah yang lebih kecil. Masalah yang ingin diselesaikan adalah membuat Voxel yang mirip dengan Mesh. Voxel sendiri adalah elemen diskrit yang merepresentasikan objek 3D (biasanya berbentuk balok atau kubus). Konsep pembagian yang ingin diterapkan adalah

- Jika terdapat komponen "mesh" di dalam Voxel, lakukan "pembagian" dan cari lokasi "mesh" yang lebih akurat
- Jika lokasi sudah dirasa akurat, anggap Voxel sekarang sebagai leaf dan hentikan iterasi
- Gabungkan leaf yang terbentuk dalam satu file

Untuk menghitung seberapa akurat, digunakan teknik Axis-Aligned Bounding Boxes (AABB) sederhana. Program akan melakukan konversi segitiga mesh menjadi AABB (seperti collider). Jika Mesh dan Voxel berinteraksi (mesh berada di dalam voxel atau intersect dengan voxel).

b. Implementasi

Kode sumber: github.com/Nyanist/Tucil2_13524047

i. AABB

Cek bounding-box pake algs dari sini (AABB aja, ngga sampe SAT).

```
using System;

namespace obj_voxelizer
{
    public class AABB
    {
        public Vectorz min;
        public Vectorz max;

        public AABB(Vectorz min, Vectorz max)
        {
            this.min = min;
            this.max = max;
        }

        public AABB(Vectorz vertices1, Vectorz vertices2, Vectorz vertices3)
        {
            this.min = Vectorz.Min(Vectorz.Min(vertices1, vertices2), vertices3);
            this.max = Vectorz.Max(Vectorz.Max(vertices1, vertices2), vertices3);
        }

    }

    public class Collision
    {
        // A intersects B
        public static bool IntersectAABB(AABB a, AABB b)
        {
            return a.min.X <= b.max.X && a.max.X >= b.min.X &&
                   a.min.Y <= b.max.Y && a.max.Y >= b.min.Y &&
                   a.min.Z <= b.max.Z && a.max.Z >= b.min.Z;
        }

        // A contains B
        public static bool ContainsAABB(AABB a, AABB b)
        {
            return a.min.X <= b.min.X && a.max.X >= b.max.X &&
                   a.min.Y <= b.min.Y && a.max.Y >= b.max.Y &&
                   a.min.Z <= b.min.Z && a.max.Z >= b.max.Z;
        }
    }
}
```

ii. Mesh

Mesh terdiri dari vertices sama faces (menyesuaikan yang ada di .obj contoh)

```
using System;

namespace obj_voxelizer
{
    public class Mesh
    {
        public List<Vectorz> vertices = new List<Vectorz>();
        public List<(int, int, int)> faces = new List<(int, int, int)>();

        public Mesh(List<Vectorz> vertices, List<(int, int, int)> faces)
        {
            this.vertices = vertices;
            this.faces = faces;
```

```
        }

        public AABB getBounds()
        {
            Vectorz min = vertices[0];
            Vectorz max = vertices[0];

            foreach (var v in vertices)
            {
                min = Vectorz.Min(min, v);
                max = Vectorz.Max(max, v);
            }

            return new AABB(min, max);
        }
    }
}
```

iii. ObjectIO

Import dan export.obj files. Import dengan cara parsing cek per line (valid only for v and f) (kalo di contoh ada vn diskip). Untuk export, program ObjectIO juga melakukan konversi Nodes menjadi Vertices dan Faces. Namun, konversi masih dilakukan manual (2 segitiga di satu sisi agar berbentuk kubus). Voxel dibentuk dari 8 titik sudut yang dibentuk dari kombinasi bound AABB Node.

```
using System;
using System.Collections.Generic;
using System.IO;

namespace obj_voxelizer
{
    public class ObjectIO
    {
        public static List<Vectorz> vertices = new List<Vectorz>();
        public static List<(int, int, int)> faces = new List<(int, int, int)>();
        public static Mesh Import(string filePath)
        {
            Console.WriteLine($"Importing OBJ file from: {filePath}");

            var lines = File.ReadAllLines(filePath);

            foreach (var line in lines)
            {
                switch (line)
                {
                    case var l when l.StartsWith("v "):
                        // Parse vertex line
                        var parts = l.Split(' ', StringSplitOptions.RemoveEmptyEntries);
                        if (parts.Length >= 4)
                        {
                            float x = float.Parse(parts[1]);
                            float y = float.Parse(parts[2]);
                            float z = float.Parse(parts[3]);
                            vertices.Add(new Vectorz(x, y, z));
                        }
                        break;
                    case var l when l.StartsWith("f "):
                        // Parse face line
                        parts = l.Split(' ', StringSplitOptions.RemoveEmptyEntries);
```

```
                        if (parts.Length >= 4)
                        {
                            int v1 = int.Parse(parts[1].Split(' ')[0]) - 1;
                            int v2 = int.Parse(parts[2].Split(' ')[0]) - 1;
                            int v3 = int.Parse(parts[3].Split(' ')[0]) - 1;
                            faces.Add((v1, v2, v3));
                        }
                        break;
                    default:
                        // Handle other line types
                        // Skip empty or invalid lines
                        break;
                }
            }
            Mesh ret = new Mesh(vertices, faces);
            return ret;
        }

        public static void Export(List<Octree> leafNodes, string outputPath)
        {
            Console.WriteLine($"Exporting voxelized data to: {outputPath}");
            using (StreamWriter writer = new StreamWriter(outputPath))
            {
                int cnt = 0;
                foreach (var node in leafNodes)
                {
                    // redundant voxel algs, tapi gatau mau gimana lagi hehe
                    // dia selalu buat node dengan ukuran maxDepth, jadi lama
                    // harusnya bisa dioptimalisasi pake skema 'merge' dari merge-sort, tapi
gitu deh

                    writer.WriteLine($"v {node.bounds.min.X} {node.bounds.min.Y}
{node.bounds.min.Z}"); // 0 0 0
                    writer.WriteLine($"v {node.bounds.min.X} {node.bounds.min.Y}
{node.bounds.max.Z}"); // 0 0 1
                    writer.WriteLine($"v {node.bounds.min.X} {node.bounds.max.Y}
{node.bounds.min.Z}"); // 0 1 0
                    writer.WriteLine($"v {node.bounds.min.X} {node.bounds.max.Y}
{node.bounds.max.Z}"); // 0 1 1
                    writer.WriteLine($"v {node.bounds.max.X} {node.bounds.min.Y}
{node.bounds.min.Z}"); // 1 0 0
                    writer.WriteLine($"v {node.bounds.max.X} {node.bounds.min.Y}
{node.bounds.max.Z}"); // 1 0 1
                    writer.WriteLine($"v {node.bounds.max.X} {node.bounds.max.Y}
{node.bounds.min.Z}"); // 1 1 0
                    writer.WriteLine($"v {node.bounds.max.X} {node.bounds.max.Y}
{node.bounds.max.Z}"); // 1 1 1

                    writer.WriteLine($"f {cnt+1} {cnt+2} {cnt+3}");
                    writer.WriteLine($"f {cnt+2} {cnt+3} {cnt+4}");
                    writer.WriteLine($"f {cnt+5} {cnt+6} {cnt+7}");
                    writer.WriteLine($"f {cnt+6} {cnt+7} {cnt+8}");

                    writer.WriteLine($"f {cnt+1} {cnt+2} {cnt+5}");
                    writer.WriteLine($"f {cnt+2} {cnt+5} {cnt+6}");
                    writer.WriteLine($"f {cnt+3} {cnt+4} {cnt+7}");
                    writer.WriteLine($"f {cnt+4} {cnt+7} {cnt+8}");

                    writer.WriteLine($"f {cnt+1} {cnt+3} {cnt+5}");
                    writer.WriteLine($"f {cnt+3} {cnt+5} {cnt+7}");
                    writer.WriteLine($"f {cnt+2} {cnt+4} {cnt+6}");
                    writer.WriteLine($"f {cnt+4} {cnt+6} {cnt+8}");

                    cnt += 8;
                }
            }
        }
```

```
    }
}
```

iv. Octree

Memiliki field AABB, isLeaf, child, depth, EPSILON dan method Constructor, GetChildren, dan isAdjacent(awalnya ini mau dipake buat merge, jadi merge yang adjacent jadi satu voxel tapi gajadi hehe). GetChildren() diimplementasi dengan membentuk Octree yang memiliki AABB lebih kecil. Pembagian menjadi 8 dengan ukuran sama (menggunakan min, center, dan max).

```
using System;

namespace obj_voxelizer
{
    public class Octree
    {
        public AABB bounds;
        public bool
        isLeaf = true; // buat ngecek 'fit' ke mesh, tapi gajadi di implement hehe
        public Octree[] child = new Octree[8];
        public int depth;
        // public Mesh mesh;
        public static double EPSILON = 1e-6;

        public Octree(AABB bounds, int depth = 0)
        {
            this.bounds = bounds;
            this.depth = depth;
        }

        public void GetChildren()
        {
            Vectorz center = (bounds.min + bounds.max) / 2;
            Vectorz min = bounds.min;
            Vectorz max = bounds.max;

            // min -> center -> max
            // prev (minX, minY, minZ) -> (maxX, maxY, maxZ)
            // prev (minX, minY, minZ) -> (centerX, centerY, centerZ)
            // prev (centerX, centerY, centerZ) -> (maxX, maxY, maxZ)
            // prev's combination

            child[0] = new Octree(new AABB(new Vectorz(min.X   , min.Y   , min.Z   ), new
Vectorz(center.X, center.Y, center.Z)), depth + 1);
            child[1] = new Octree(new AABB(new Vectorz(center.X, min.Y   , min.Z   ), new
Vectorz(max.X   , center.Y, center.Z)), depth + 1);
            child[2] = new Octree(new AABB(new Vectorz(min.X   , center.Y, min.Z   ), new
Vectorz(center.X, max.Y   , center.Z)), depth + 1);
            child[3] = new Octree(new AABB(new Vectorz(center.X, center.Y, min.Z   ), new
Vectorz(max.X   , max.Y   , center.Z)), depth + 1);
            child[4] = new Octree(new AABB(new Vectorz(min.X   , min.Y   , center.Z), new
Vectorz(center.X, center.Y, max.Z   )), depth + 1);
            child[5] = new Octree(new AABB(new Vectorz(center.X, min.Y   , center.Z), new
Vectorz(max.X   , center.Y, max.Z   )), depth + 1);
            child[6] = new Octree(new AABB(new Vectorz(min.X   , center.Y, center.Z), new
Vectorz(center.X, max.Y   , max.Z   )), depth + 1);
            child[7] = new Octree(new AABB(new Vectorz(center.X, center.Y, center.Z), new
Vectorz(max.X   , max.Y   , max.Z   )), depth + 1);
```

```
            isLeaf = false;
        }

        public bool isAdjacent(Octree other)
        {
            return ((Math.Abs(bounds.min.X - other.bounds.max.X) < EPSILON ||
Math.Abs(bounds.max.X - other.bounds.min.X) < EPSILON) &&
                    bounds.min.Y < other.bounds.max.Y && bounds.max.Y > other.bounds.min.Y &&
                    bounds.min.Z < other.bounds.max.Z && bounds.max.Z > other.bounds.min.Z) ||
                    ((Math.Abs(bounds.min.Y - other.bounds.max.Y) < EPSILON ||
Math.Abs(bounds.max.Y - other.bounds.min.Y) < EPSILON) &&
                    bounds.min.X < other.bounds.max.X && bounds.max.X > other.bounds.min.X &&
                    bounds.min.Z < other.bounds.max.Z && bounds.max.Z > other.bounds.min.Z) ||
                    ((Math.Abs(bounds.min.Z - other.bounds.max.Z) < EPSILON ||
Math.Abs(bounds.max.Z - other.bounds.min.Z) < EPSILON) &&
                    bounds.min.X < other.bounds.max.X && bounds.max.X > other.bounds.min.X &&
                    bounds.min.Y < other.bounds.max.Y && bounds.max.Y > other.bounds.min.Y);
        }

        // public void Subdivide(int maxDepth)
        // {
        //     if (depth > maxDepth || depth < 0)
        //         return;

        //     GetChildren();

        //     for (int i = 0; i < 8; i++)
        //     {
        //         child[i].depth = this.depth + 1;
        //         if (child[i].IntersectsMesh())
        //             child[i].Subdivide(maxDepth);
        //         else
        //             child[i].isLeaf = false;
        //     }
        // }

        // public bool IntersectsMesh()
        // {
        //     // Check if mesh intersects with octree
        //     foreach (var face in mesh.faces)
        //     {
        //         AABB triangleBounds = new AABB(
        //             mesh.vertices[face.Item1],
        //             mesh.vertices[face.Item2],
        //             mesh.vertices[face.Item3]
        //         );

        //         if (Collision.IntersectAABB(triangleBounds, bounds))
        //             return true;
        //     }
        //     return false;
        // }

    }
}
```

v. Program

Alur main input path .obj → max depth → process → stat → path export → export

```
// Main
using System;
```

```
namespace obj_voxelizer
{
    class Program
    {
        static void Main()
        {
            try
            {
                Console.WriteLine("Masukkan path menuju file OBJ (full path):");
                string? ObjPath = Console.ReadLine();
                ObjPath = ObjPath?.Trim();

                if (string.IsNullOrEmpty(ObjPath))
                    throw new Exception("File path kosong.");

                Mesh mesh = ObjectIO.Import(ObjPath);
                if (mesh.vertices.Count == 0 || mesh.faces.Count == 0)
                    throw new Exception("Import gagal. File kosong atau invalid.");

                Console.WriteLine($"Berhasil memuat data.");
                Console.WriteLine($"Mesh: {mesh.vertices.Count} vertices, {mesh.faces.Count}
faces.");

                Console.WriteLine("Masukkan kedalaman maksimum (default 8):"); // jujur udah
keburu integrate ini, maaf kalo harusnya gaada wkwkwk

                string? inputDepth = Console.ReadLine();
                int maxDepth = Voxelizer.DEFAULT_DEPTH; // redundant si tapi yaudah
                if (!string.IsNullOrEmpty(inputDepth) && int.TryParse(inputDepth, out int
parsedDepth) && parsedDepth >= 0)
                    maxDepth = parsedDepth;

                maxDepth = Math.Min(maxDepth, Voxelizer.MAX_DEPTH); // mmf biar ga kelamaan
                Console.WriteLine($"kedalaman maksimal: {maxDepth}");

                Console.WriteLine("Memulai voxelization...");
                Voxelizer voxelizer = new Voxelizer(mesh, maxDepth);
                DateTime startTime = DateTime.Now;
                voxelizer.Voxelize();
                DateTime endTime = DateTime.Now;
                TimeSpan duration = endTime - startTime;

                Console.WriteLine($"Program selesai dalam {duration.TotalMilliseconds} ms.");

                voxelizer.GetStats();

                Console.WriteLine($"Masukkan output path untuk voxelized OBJ file (atau Enter
untuk default path):");
                string? outputPath = Console.ReadLine();
                outputPath = outputPath?.Trim();
                if (string.IsNullOrEmpty(outputPath))
                    outputPath = Path.ChangeExtension(ObjPath, "_voxels.obj");

                ObjectIO.Export(Voxelizer.GetLeaf(), outputPath);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error: {ex.Message}");
            }
        }
    }
}
```

vi. Vectorz

Basically vector 3d dan operasinya

```
using System;

namespace obj_voxelizer
{
    public struct Vectorz
    {
        public float X, Y, Z;

        public Vectorz(float x, float y, float z)
        {
            X = x;
            Y = y;
            Z = z;
        }

        public static Vectorz operator +(Vectorz a, Vectorz b)
        {
            return new Vectorz(a.X + b.X, a.Y + b.Y, a.Z + b.Z);
        }

        public static Vectorz operator -(Vectorz a, Vectorz b)
        {
            return new Vectorz(a.X - b.X, a.Y - b.Y, a.Z - b.Z);
        }

        public static Vectorz operator *(Vectorz a, float s)
        {
            return new Vectorz(a.X * s, a.Y * s, a.Z * s);
        }

        public static Vectorz operator /(Vectorz a, float s)
        {
            return new Vectorz(a.X / s, a.Y / s, a.Z / s);
        }

        public float Length()
        {
            return (float)Math.Sqrt(this.X * this.X + this.Y * this.Y + this.Z * this.Z);
        }

        public float LengthSquared() {
            return this.X * this.X + this.Y * this.Y + this.Z * this.Z;
        }

        public Vectorz Normalized()
        {
            float len = Length();
            return len > 0 ? this / len : this;
        }

        public static float Dot(Vectorz a, Vectorz b)
        {
            return a.X * b.X + a.Y * b.Y + a.Z * b.Z;
        }

        public static Vectorz Cross(Vectorz a, Vectorz b)
        {
            return new Vectorz(
                a.Y * b.Z - a.Z * b.Y,
                a.Z * b.X - a.X * b.Z,
                a.X * b.Y - a.Y * b.X
            );
        }

        public static Vectorz Min(Vectorz a, Vectorz b)
        {
            return new Vectorz(
```

```
                Math.Min(a.X, b.X),
                Math.Min(a.Y, b.Y),
                Math.Min(a.Z, b.Z)
            );
        }

        public static Vectorz Max(Vectorz a, Vectorz b)
        {
            return new Vectorz(
                Math.Max(a.X, b.X),
                Math.Max(a.Y, b.Y),
                Math.Max(a.Z, b.Z)
            );
        }

    }
}
```

vii. Voxelizer

Inti algsnya ada di sini (sort of). Voxelizer punya mesh dan octree. Voxelizer membantu Octree menyesuaikan dirinya dengan mesh. Salah satu caranya adalah dengan metode Voxelize. Metode ini memanggil fungsi GetChildren() milik Octree jika dibutuhkan (jujur masih binggung biar dia bisa berenti sendiri gimana, jadi awalnya mau ngemerge hasil akhir aja, tapi gajadi juga hehe). Metode ini juga menghitung node yang dijelajahi dan ditolak. Node akan dijelajahi jika memenuhi metode Validate(). Metode ini mengembalikan true jika AABB mesh berada dalam AABB octree atau AABB mesh dan AABB octree intersect. GetStats() basically implementasi keluaran sesuai dengan yang ada di spesifikasi. (vertices dan faces dihitung di sini, karena pembagiannya pasti di Export nanti)

```
using System;
using System.Reflection.Metadata;

namespace obj_voxelizer
{
    public class Voxelizer
    {
        public Mesh mesh;
        public Octree octree;
        public int maxDepth;
        private List<Octree> potentialNodes = new List<Octree>();
        public const int MAX_INT = 1000000007;
        public const int DEFAULT_DEPTH = 8;
        public const int MAX_DEPTH = 20;
        private static int[] nodeExCount = new int[MAX_INT];
        private static int[] nodeUnCount = new int[MAX_INT];
        private static int[] leafCount = new int[MAX_INT];
        private static List<Octree> leafNodes = new List<Octree>();

        public Voxelizer(Mesh mesh, int maxDepth = DEFAULT_DEPTH)
        {
            this.mesh = mesh;
            this.octree = new Octree(mesh.getBounds());
```

```
            this.maxDepth = maxDepth;
        }

        public Voxelizer(Mesh mesh, Octree octree, int maxDepth = DEFAULT_DEPTH)
        {
            this.mesh = mesh;
            this.octree = octree;
            this.maxDepth = maxDepth;
        }

        public void Voxelize()
        {
            int depth = this.octree.depth;
            // GetChildren()?
            switch (depth)
            {
                case int d when d == maxDepth: // mentok
                    nodeExCount[depth]++;
                    leafCount[depth]++;
                    leafNodes.Add(this.octree);
                    break;
                case int d when d > maxDepth || d < 0: // invalid
                    break;
                default:
                    nodeExCount[depth]++; // explored
                    octree.GetChildren();
                    foreach (var child in octree.child)
                    {
                        // Check if child intersect or contains mesh
                        if (this.Validate(child))
                        {
                            Voxelizer childVoxelizer = new Voxelizer(mesh, child, maxDepth);
                            childVoxelizer.Voxelize(); // Asumsi pasti ada minimal 1 child yang
valid
                            // potentialNodes.Add(child);
                        }
                        else
                            nodeUnCount[child.depth]++; // unexplored
                    }
                    break;
            }
            // Check potential nodes, turn this into leaf or check children
            // switch (potentialNodes.Count)
            // {
            //     case int cnt when cnt == 8: // why bother?
            //         this.octree.isLeaf = true;
            //         leafNodes.Add(this.octree);
            //         break;
            //     default: // lemme check this for a second
            //         foreach (var node in potentialNodes)
            //         {
            //             Voxelizer childVoxelizer = new Voxelizer(mesh, node, maxDepth);
            //             childVoxelizer.Voxelize();
            //         }
            //         break;
            // }


        }

        public bool Validate(Octree node)
        {
            foreach (var faces in mesh.faces)
            {
                AABB triangle = new AABB(mesh.vertices[faces.Item1],
                                         mesh.vertices[faces.Item2],
                                         mesh.vertices[faces.Item3]);
```

```
                if (
                    Collision.IntersectAABB(triangle, node.bounds) ||
                    Collision.ContainsAABB(node.bounds, triangle))
                    return true; // AABB intersect or contain triangle
            }
            return false;
        }

        public void GetStats()
        {
            Console.WriteLine("--Stats");

            Console.WriteLine($"Banyaknya voxel yang terbentuk: {leafNodes.Count}");
            Console.WriteLine($"Banyaknya vertex yang terbentuk: {leafNodes.Count * 8}");
            Console.WriteLine($"Banyaknya faces yang terbentuk: {leafNodes.Count * 12}");

            Console.WriteLine("Banyaknya node yang terbentuk per depth:");
            for (int i = 0; i <= maxDepth; i++)
                Console.WriteLine($"{i}: {nodeExCount[i]}");

            Console.WriteLine("Banyaknya node yang tidak dieksplorasi per depth:");
            for (int i = 0; i <= maxDepth; i++)
                Console.WriteLine($"{i}: {nodeUnCount[i]}");
        }

        public static List<Octree> GetLeaf()
        {
            return leafNodes;
        }
    }
}
```

2. Eksperimen

Test1
Input
\src\example-object\pumpkin.obj 6
Output

```
Berhasil memuat data.
Mesh: 5002 vertices, 10000 faces.
Masukkan kedalaman maksimum (default 8):
6
kedalaman maksimal: 6
Memulai voxelization...
Program selesai dalam 78363.0801 ms.
--Stats
Banyaknya voxel yang terbentuk: 37974
Banyaknya vertex yang terbentuk: 303792
Banyaknya faces yang terbentuk: 455688
Banyaknya node yang terbentuk per depth:
0: 1
1: 8
2: 56
3: 312
4: 1479
5: 7275
6: 37974
Banyaknya node yang tidak dieksplorasi per depth:
0: 0
1: 0
2: 8
3: 136
4: 1017
5: 4557
6: 20226
Masukkan output path untuk voxelized OBJ file (atau Enter untuk default path):
```

Test2
Input
\src\example-object\pumpkin.obj
Output

```
Berhasil memuat data.
Mesh: 5002 vertices, 10000 faces.
Masukkan kedalaman maksimum (default 8):

kedalaman maksimal: 8
Memulai voxelization...
Program selesai dalam 2318428.0098 ms.
--Stats
Banyaknya voxel yang terbentuk: 1433904
Banyaknya vertex yang terbentuk: 11471232
Banyaknya faces yang terbentuk: 17206848
Banyaknya node yang terbentuk per depth:
0: 1
1: 8
2: 56
3: 312
4: 1479
5: 7275
6: 37974
7: 220865
8: 1433904
Banyaknya node yang tidak dieksplorasi per depth:
0: 0
1: 0
2: 8
3: 136
4: 1017
5: 4557
6: 20226
7: 82927
8: 333016
Masukkan output path untuk voxelized OBJ file (atau Enter untuk default path):
```

Test3
Input
\src\example-object\teapot.obj 6
Output

```
Berhasil memuat data.
Mesh: 530 vertices, 1024 faces.
Masukkan kedalaman maksimum (default 8):
6
kedalaman maksimal: 6
Memulai voxelization...
Program selesai dalam 5221.3382 ms.
--Stats
Banyaknya voxel yang terbentuk: 50651
Banyaknya vertex yang terbentuk: 405208
Banyaknya faces yang terbentuk: 607812
Banyaknya node yang terbentuk per depth:
0: 1
1: 8
2: 60
3: 272
4: 1446
5: 8099
6: 50651
Banyaknya node yang tidak dieksplorasi per depth:
0: 0
1: 0
2: 4
3: 208
4: 730
5: 3469
6: 14141
Masukkan output path untuk voxelized OBJ file (atau Enter untuk default path):
```

Test4
Input
\src\example-object\cow.obj 6
Output

```
Berhasil memuat data.
Mesh: 4583 vertices, 5804 faces.
Masukkan kedalaman maksimum (default 8):
6
kedalaman maksimal: 6
Memulai voxelization...
Program selesai dalam 36227.2863 ms.
--Stats
Banyaknya voxel yang terbentuk: 32705
Banyaknya vertex yang terbentuk: 261640
Banyaknya faces yang terbentuk: 392460
Banyaknya node yang terbentuk per depth:
0: 1
1: 8
2: 50
3: 252
4: 1180
5: 5915
6: 32705
Banyaknya node yang tidak dieksplorasi per depth:
0: 0
1: 0
2: 14
3: 148
4: 836
5: 3525
6: 14615
Masukkan output path untuk voxelized OBJ file (atau Enter untuk default path):
```

Test5
Input
\src\example-object\line.obj
Output

```
Berhasil memuat data.
Mesh: 3 vertices, 1 faces.
Masukkan kedalaman maksimum (default 8):

kedalaman maksimal: 8
Memulai voxelization...
Program selesai dalam 38965.174 ms.
--Stats
Banyaknya voxel yang terbentuk: 16777216
Banyaknya vertex yang terbentuk: 134217728
Banyaknya faces yang terbentuk: 201326592
Banyaknya node yang terbentuk per depth:
0: 1
1: 8
2: 64
3: 512
4: 4096
5: 32768
6: 262144
7: 2097152
8: 16777216
Banyaknya node yang tidak dieksplorasi per depth:
0: 0
1: 0
2: 0
3: 0
4: 0
5: 0
6: 0
7: 0
8: 0
Masukkan output path untuk voxelized OBJ file (atau Enter untuk default path):
```

3. Kesimpulan dan Evaluasi

Program dapat dioptimasi dengan penambahan konsep merge pada merge-sort. Program telah berhasil membagi voxel menjadi voxel yang lebih kecil melalui Octree. Program telah berhasil melakukan dekomposisi pada permasalahan yang ada. Namun, program belum dapat melakukan generalisasi pada hasil yang mirip atau bisa digabungkan ke dalam satu Voxel. Masalah lain yang ditemukan adalah, karena ekstraksi dilakukan dengan algoritma yang tidak sangkil, program menghasilkan dokumen dengan ukuran yang besar.

test1.txt	03/28/26 00:12	Text Document	1 KB
test1_voxels.obj	03/28/26 00:16	OBJ File	20,333 KB
test2.txt	03/28/26 00:18	Text Document	1 KB
test2_voxels.obj	03/28/26 00:58	OBJ File	837,692 KB
test3.txt	03/28/26 00:16	Text Document	1 KB
test3_voxels.obj	03/28/26 01:05	OBJ File	27,051 KB
test4.txt	03/28/26 01:06	Text Document	1 KB
test4_voxels.obj	03/28/26 01:08	OBJ File	17,606 KB
test5.txt	03/27/26 23:45	Text Document	1 KB
test5_voxels.obj	03/28/26 01:12	OBJ File	3,655,136 KB

Saran optimalisasi kedepannya dapat menerapkan algoritma untuk memangkas keluaran dan iterasi program. Program dapat dibuat agar melakukan divide dengan heuristik yang lebih spesifik. Pendekatan dengan Separated Axis Theorem (SAT) juga dapat diimplementasikan untuk menggantikan AABB karena algoritma SAT lebih sangkil. Program yang lebih optimal diharapkan mampu membentuk vertices dan faces dengan lebih efektif juga. Face hanya terbentuk untuk sisi yang benar-benar leaf. Voxel juga diharapkan dapat digabungkan jika memiliki batas dan gradien yang mirip.

4. Penutup

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	x	
2	Program berhasil dijalankan	x	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	x	
4	Program informasi-informasi atau report	x	
5	Program dapat menerima masukan .obj, dan mengembalikan output	x	
6	[Bonus] Program menggunakan *concurrency*		x

7	[Bonus] Program dapat menampilkan visualisasi .obj		x

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Jordan Ferimeison